ledger-core — Architecture & Trade-offs

Yazan Tommalieh · November 2026 · Companion to the Part 1 repository (`ledger-core`, Python, append-only, value-dated)

The core has one primitive: `Ledger.balance(account, value_date_upto, as_of_posting_day)`, a sum over an immutable log where every record carries both the day it was posted and the day it is effective. There is no running balance. Hold state, the once-per-day fee latch and the interest schedule are all queries over that log. That is what makes the design correct under backdated entries, and it is also the root of every scaling problem below.

---

1. Append-only at scale

What breaks first at 100× volume is the end-of-day close, not storage. `Engine.end_of_day_fees` re-examines every day from 1 to today for every account, because a backdated entry can turn an already-closed day negative; each examination is a `balance()` call that scans the whole entry list. One close therefore costs roughly *days × accounts × entries*, and entries themselves grow with *days × accounts*. The close is quadratic in history and quadratic in accounts. At 100× accounts it is ~10,000× the work, and a close is the one job with a hard deadline (the next business day). The interest schedule has the same shape: `interest_schedule` rebuilds every day's accrual from scratch on every call, and the report calls it once per account per day. Nothing goes *wrong* at scale — there is no cached state to go stale, which is the point of the design — it just stops finishing on time.

Where unbounded state accumulates. Three lists that are never compacted and must not be, because they are the audit record: `_entries`, `_auths`, `_errors`. One that could be and is not: `_seen_event_ids`, the idempotency set, which grows by one per event for the life of the process. Every lookup is a linear scan over one of those lists (`auth`, `settlement_of`, `fee_for`, `entries_for_event`, `active_holds`). And one piece of cross-account state, the global `_seq` counter, which quietly makes the whole engine a single writer: nothing in the data model couples two accounts (there are no transfers), but the sequence does.

The cheapest structural change that defers this: close the period. Introduce a value-date horizon *H* (the last closed business day, or the last statement cycle). `append_entry` refuses any entry with `value_date < H` — one comparison next to the existing future-date check — and at each close the engine writes one snapshot row per account: the balance, held amount and fee-latch state as of *H*. Every balance query becomes *snapshot(H) + scan of entries with value_date > H*; the fee re-walk covers *today − H* days instead of all of them; the journal before *H* can go cold; the idempotency set only needs ids newer than *H*. The journal stays append-only and the snapshot is derivable from it, so nothing about auditability changes. It is cheaper than indexing because it bounds the *problem* rather than speeding up the scan, and it is the shape finance and the regulator want anyway: a period that has been reported and will not move. What it defers rather than solves: the journal is still unbounded by design (five-year retention), and account-level parallelism still needs per-account sequences in place of the global one. Those are the second and third changes, in that order. An index on `(account, value_date)` is the fourth; by then it is an optimisation, not a rescue.

---

2. Value-dated entries in production

A value-dated entry is a licence to rewrite history. In the Part 1 stream one backdated debit restated three closed days, produced three fees in a single close and declined an authorization that would otherwise have been approved; a reversal the next day restated the same days again and left the fees standing. In a UAE-licensed bank each of those effects has an owner.

- **The customer and the complaints desk.** A statement already issued is now wrong, and a fee appears "for" a day on which the customer saw a positive balance. Under the CBUAE Consumer Protection Standards (Circular 8/2020) a complaint must be acknowledged within 2 business days and answered in writing within 30 business days (cl. 8.1.2.5, 8.1.3.7), with the file kept for five years (cl. 8.1.3.3(e)). Fees generated by re-assessment are the single most predictable generator of those complaints, and

the “history restated” block the engine prints per day is the material the complaints desk will need to answer them.

- **The fee framework.** Overdraft charges sit inside the CBUAE retail fee regime (Regulations 29/2011, Annexure 2 as amended in 2018: 43 services capped; any new fee or change above 5% needs ex-ante CBUAE approval in the April/October windows). An engine that can turn one backdated debit into three charges is a change in fee *behaviour*, not just an implementation detail; compliance has to sign it off and the customer agreement has to describe it.
- **Finance.** Interest is recomputed from the restated history, so a backdated entry that crosses a month-end changes already-reported fee income and accrued interest. The core shows the new number; a GL interface needs reversing and re-posting journals in the period the correction is made, which the core does not emit.
- **Audit and AML.** Every record carries its posting day and value date, so “what did we know on Day 5” is a query (`as_of_posting_day`), and the journal satisfies the five-year retention now required by Federal Decree-Law No. 10 of 2025 (in force 14 October 2025). What it does not carry is who and why: no operator, channel or reason code on a backdated entry, which is the first thing an examiner asks for.
- **The comparison worth knowing.** Under PSD2 Art. 87(3) a debit value date may not be earlier than the moment the account is debited; the Part 1 backdated debit would be unlawful as a customer payment in the EU. CBUAE does not say that in those words, but fair-treatment plus the complaint machinery arrives at the same place. The legitimate sources of backdated entries here are bank-side: scheme clearing files (card transactions clear T+1/T+2 carrying the transaction date), corrections, and charge or interest adjustments. None are customer-initiated, and the control below is built on that fact.
- **Two things the integer-day model hides.** Production value dates are business days in Gulf Standard Time with a Saturday–Sunday weekend and a cut-off time, so “value date Day 2” becomes a convention that has to be agreed. And if the core ever serves an Islamic window, “interest” is “profit” computed on a different basis; the accrual engine needs a second implementation, not a parameter.

The one control I would add before go-live: a backdating horizon with maker-checker. The core refuses any entry value-dated before the last closed day; anything that must land behind it goes through an adjustments queue where a second person approves it with a reason code, and that approval is appended to the journal as a record of its own. I choose this over the alternatives because it turns an unbounded surface (every fee, accrual, statement and reported figure for all history) into a bounded one, it produces the audit trail the examiner will ask for first (who backdated, why, who approved), and it is the condition under which automatic fee re-assessment is safe to run at all. The second control — a causal link from every generated fee to the entry that triggered it, so that reversing the cause reverses its fees — is the subject of the deliberately failing test in Part 1 and of row 4 in section 4.

---

3. Authorization lifecycle

In the model an authorization is a decision record (approved or declined); its state — DECLINED, ACTIVE, SETTLED — is derived from whether a settlement entry references it. The only ending the model implements fully is the matching settlement. Every other ending, what it stands for in the card world, and what I would make the system do:

Ending	In the model today	Real-world scenario	Behaviour I would mandate
Declined at request	Recorded with available-before/after; no hold ever exists; never revisited (AMBIGUITIES §11).	Insufficient available funds; in practice also fraud/velocity rules, blocked MCC, bad PIN or CVV.	Decline is final and carries a reason code. A later backdated credit does not re-open it; the merchant re-requests if it wants to.
Duplicate request	Same auth id again → rejected as `DUPLICATE_AUTH`; second record never created.	Network retry, replayed message after a timeout.	Idempotent: answer with the original decision, identical response, no second hold. The model records an error; production should simply repeat the first answer.

Ending	In the model today	Real-world scenario	Behaviour I would mandate
Never settled (expiry)	Hold stays ACTIVE for ever; the report flags “unsettled at window end”; no expiry exists.	Abandoned pre-authorization: hotel, car rental, fuel, an online order cancelled without a reversal; merchant outage.	Expire per scheme rules — Visa 7 days by default, 31 for lodging, vehicle rental and cruise; Mastercard 7 days for a final authorization, 30 for a pre-authorization, extendable. Expiry is appended as an event, the hold returns to available, EXPIRED is terminal.
Reversed by the merchant	No such event type.	Order cancelled before capture; partial reversal after a partial shipment.	New event: full reversal → VOIDED, partial reversal → hold reduced. Only the merchant/acquirer can originate it; the issuer never voids on its own.
Late presentment / no authorization	Rejected, nothing posted (Part 1 criterion 4).	Clearing arrives after expiry or with no matching authorization: stand-in processing while the issuer was down, offline chip approval below floor limit, a force-post.	Post the debit flagged `NO_AUTH/LATE` so the ledger agrees with money the scheme has already settled, and raise it to disputes. The issuer's remedy is a chargeback (late presentment), not refusal.
Settled for a different amount	Partial: posted, whole hold released (§26). Over: posted with an `OVER_SETTLEMENT` warning (§16).	Final bill below the estimate; tips, fuel and hotel incidentals above it.	Partial capture releases the remainder unless the merchant flagged multi-capture, in which case the residual stays held at the reduced amount until expiry. Over-capture allowed within the scheme's tolerance for that merchant category; the excess beyond it is treated as late presentment.
Settlement later reversed	Funds returned by a reversal entry; the authorization stays SETTLED, no hold is reinstated (§29).	Refund; chargeback.	A refund is a new credit, never a reinstated hold. A chargeback is its own workflow with its own entries and provisional-credit rules.
Account state change	Not modelled; no account lifecycle.	Fraud freeze, legal hold, death, dormancy (CBUAE Dormant Accounts Regulation).	A freeze blocks new authorizations but does not release existing holds; a legal hold keeps them until instructed; closure releases them and routes any late clearing to disputes.
Incremental / extended	Not modelled; a second request with the same id is a duplicate.	Hotel stay extended; rental returned late.	Incremental authorization amends the existing hold and resets its expiry; it is one authorization with a history, not two.

4. What I cut and why

Simplification	Why it stayed in scope	Production risk it defers
In-memory lists, linear scans, one global sequence.	The brief forbids persistence; it keeps the balance primitive readable.	Quadratic close (§1); single writer; nothing survives a restart — including the idempotency set, so a crash mid-stream can double-post on replay.
Single-sided entries: no double-entry, no contra account, no transfers between accounts.	The stream has no transfers.	No debits-equal-credits invariant to prove integrity; nothing to reconcile to the general ledger; a transfer needs an atomic two-entry unit the store does not have.
No hold expiry, no merchant reversal, no incremental authorization, no multi-capture.	None occur in the stream.	Holds lock customer funds indefinitely; the most common real-world authorization ending (a merchant void) cannot be represented. Section 3.

Simplification	Why it stayed in scope	Production risk it defers
Fees carry no causal link to the entry that triggered them; no fee reversal.	The spec defines when to assess, never when to unwind. This is the deliberately failing test.	A reversed bank error leaves the customer paying for an overdraft that, by value date, never happened — a complaint, then a finding. Fix: a `trigger_seq` on every derived entry and a compensating fee reversal when the trigger is reversed, automatic or queued per product rule.
Authorizations decided on the ledger as known at that moment; approvals never revisited after a backdated debit.	That is how card networks behave; a ledger that claws back approvals is unusable for merchants.	Exposure from backdated debits sits with the bank; nothing flags “approved on a balance later restated” for risk review.
Debits are never declined; no overdraft limit; a flat fee per day with no cap.	The spec gives no limit and one fee amount.	Unlimited overdraft; fee stacking (one backdated debit → three charges). Production needs an overdraft line per account and a per-cycle fee cap, both of which the CBUAE fee framework will ask about.
Interest: simple, daily, positive balances only, one capitalisation at window end; no debit interest, tiers or Islamic profit model.	Spec wording.	Wrong product behaviour for any real account; a restated accrual across a reporting period is a prior-period adjustment with no journal to carry it.
Days are integers: no calendar, weekends, holidays, cut-offs or time zones.	The brief's six days are abstract.	Value-date conventions over weekends and holidays, the Gulf-time day boundary, and cut-off disputes are all undefined.
Future value dates refused outright.	Removes the ambiguity in “balance as of day d”.	Scheduled and standing payments cannot be represented; needs an “exclude future-dated” rule in place of a refusal.
Backdating is unbounded: any value date from Day 1 is accepted.	The exercise depends on it.	Full-history restatement on every backdated entry; covered by the horizon control (§2).
Structural faults raise and stop the replay (precision, unknown account, duplicate id, non-positive amount).	Right for a test harness: a malformed stream should fail loudly.	One malformed message halts production; needs a dead-letter queue and continue.
Rejections are recorded and nothing else.	The brief asks for errors to be printed.	A genuine force-post is logged and never reaches disputes; money the scheme has already settled goes unaccounted.
No operator, channel, reason code or external reference on any entry.	The stream carries none.	Audit can show what and when but not who or why; reconciliation against scheme clearing files is impossible.
Only an AED overdraft fee is configured; an overdrawn BHD account produces an error.	The spec gives one currency's fee and no FX rule.	Silent under-charging if the error feed is not monitored; needs per-currency product configuration.
HALF_UP, rounded daily then summed.	Spec wording; no ties occur in the stream.	Product or finance may mandate round-once or banker's rounding; the difference is fils, but it must match the customer agreement.
No account lifecycle, customer or product objects.	Out of scope.	Freeze, closure and dormancy behaviour for holds (section 3, row 8) is undefined.

The honest summary: everything is a query over an immutable log, which makes the core right and slow. The production path keeps the log and adds, in order, a horizon with snapshots, causal links on derived entries, and the authorization events the card world actually sends.

References: CBUAE Consumer Protection Standards (Circular 8/2020) Art. 8; CBUAE Regulations 29/2011 Annexure 2 (fee caps, amended June 2018); Federal Decree-Law No. 10 of 2025 on AML/CFT; Directive (EU) 2015/2366 Art. 87; Visa and Mastercard authorization-hold timeframes as published by issuer processors. Prepared with Claude (Anthropic) as a drafting partner, consistent with the Part 1 WORKLOG disclosure.